



**SAPIENZA**  
UNIVERSITÀ DI ROMA

## Sapienza-DC: progettazione e sviluppo di un sistema RAG domain-aware

Implementazione di un chatbot multi-dominio basato su Retrieval-Augmented Generation

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica (I3S)  
Dipartimento di Ingegneria Informatica Automatica e Gestionale (DIAG)  
Corso di Laurea in Ingegneria Informatica e Automatica

**Fabio Pastore**

Matricola 2129632

Relatore

Prof. Roberto Navigli

Anno Accademico 2025/2026

---

**Sapienza-DC: progettazione e sviluppo di un sistema RAG domain-aware**

Tesi di laurea triennale. Sapienza Università di Roma

© 2026 Fabio Pastore. Tutti i diritti riservati

Questa tesi è stata composta con L<sup>A</sup>T<sub>E</sub>X e la classe Sapthesis.

Email dell'autore: [pastore.2129632@studenti.uniroma1.it](mailto:pastore.2129632@studenti.uniroma1.it)

*To my family and friends.*



## Sommario

Nella seguenti tesi viene descritta la realizzazione di Sapienza-DC, un chatbot RAG multi-dominio capace di rispondere alle richieste degli utenti mediante analisi e validazione di informazioni estratte da pagine web, fornendo risposte coerenti affiancate da un punteggio di affidabilità nonché dalle fonti utilizzate dall’LLM per rispondere.

I capitoli contenuti in questo elaborato tratteranno gli obiettivi fondamentali di Sapienza-DC e la raccolta dei requisiti di progetto (Cap. I); l’analisi delle tecnologie nonché metodologie utilizzate (Cap. II), la progettazione (Cap. III), l’architettura del sistema (Cap. IV), la fase di implementazione e gli sviluppi del progetto, proposti in ordine cronologico (Cap. V); un breve excursus sulle ottimizzazioni introdotte, la sicurezza della pipeline dinanzi a tentativi di prompt injection e una valutazione qualitativa, sotto forma di benchmark, del sistema complessivo (Cap. VI). Al termine della trattazione verranno invece effettuate alcune considerazioni sul lavoro svolto, sui traguardi raggiunti e su possibili sviluppi del progetto (Cap. VII).



# Indice

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                          | <b>1</b>  |
| 1.1      | Presentazione del progetto . . . . .                         | 1         |
| 1.2      | Requisiti di progetto . . . . .                              | 1         |
| <b>2</b> | <b>Analisi delle tecnologie e metodologie impiegate</b>      | <b>3</b>  |
| 2.1      | Concetti preliminari di NLP . . . . .                        | 3         |
| 2.1.1    | Large Language Model (LLM) . . . . .                         | 4         |
| 2.1.2    | Retrieval-Augmented Generation (RAG) . . . . .               | 4         |
| 2.1.3    | Bi-encoder, cross-encoder . . . . .                          | 4         |
| 2.2      | Framework, strumenti e librerie di sviluppo . . . . .        | 6         |
| 2.2.1    | FastAPI . . . . .                                            | 6         |
| 2.2.2    | LangChain . . . . .                                          | 6         |
| 2.2.3    | Hugging Face . . . . .                                       | 6         |
| 2.2.3.1  | FastEmbed . . . . .                                          | 7         |
| 2.2.3.2  | SentenceTransformers . . . . .                               | 7         |
| 2.2.4    | Ollama e llama.cpp . . . . .                                 | 7         |
| 2.2.5    | SearXNG . . . . .                                            | 8         |
| 2.2.6    | Crawl4AI . . . . .                                           | 9         |
| 2.2.7    | Jinja2 . . . . .                                             | 9         |
| 2.2.8    | TailwindCSS . . . . .                                        | 9         |
| 2.2.8.1  | Utility class . . . . .                                      | 9         |
| 2.3      | Containerizzazione ed orchestrazione . . . . .               | 9         |
| 2.3.1    | Architettura a microservizi: Docker e Docker Compose . . . . | 10        |
| 2.4      | Database relazionale . . . . .                               | 10        |
| 2.4.1    | SQL . . . . .                                                | 10        |
| 2.4.2    | MariaDB . . . . .                                            | 11        |
| 2.5      | Metodologia di sviluppo agile . . . . .                      | 11        |
| 2.6      | Paradigma client-server . . . . .                            | 11        |
| <b>3</b> | <b>Progettazione dell'applicazione</b>                       | <b>13</b> |
| 3.1      | Diagramma delle classi UML . . . . .                         | 13        |
| 3.2      | Modello ER . . . . .                                         | 15        |
| 3.2.1    | Progettazione concettuale . . . . .                          | 15        |
| 3.2.2    | Progettazione logica . . . . .                               | 16        |
| 3.2.2.1  | Ristrutturazione dello schema concettuale . . . . .          | 16        |
| 3.2.2.2  | Traduzione diretta . . . . .                                 | 17        |

---

|          |                                                                                                                              |           |
|----------|------------------------------------------------------------------------------------------------------------------------------|-----------|
| 3.2.2.3  | Ristrutturazione dello schema logico . . . . .                                                                               | 17        |
| <b>4</b> | <b>Architettura del sistema</b>                                                                                              | <b>19</b> |
| 4.1      | Architettura lato server (backend) . . . . .                                                                                 | 20        |
| 4.2      | Architettura lato client (frontend) . . . . .                                                                                | 21        |
| 4.3      | Architettura per la persistenza dei dati (MariaDB) . . . . .                                                                 | 22        |
| 4.4      | Architettura di integrazione ed estrazione di dati dal web (MWP) . .                                                         | 22        |
| <b>5</b> | <b>Implementazione della pipeline RAG</b>                                                                                    | <b>25</b> |
| 5.1      | Processamento preliminare della query dell'utente e disambiguazione                                                          | 25        |
| 5.2      | Assegnazione della query a un dominio e fallback . . . . .                                                                   | 26        |
| 5.3      | Ricerca degli URL e parsing delle pagine web . . . . .                                                                       | 27        |
| 5.3.1    | DDGS - Duck Duck Go Search . . . . .                                                                                         | 27        |
| 5.3.2    | SearXNG e qwant . . . . .                                                                                                    | 27        |
| 5.3.3    | Custom web scraper per Startpage.com . . . . .                                                                               | 27        |
| 5.3.4    | Estrazione dei dati dagli URL selezionati . . . . .                                                                          | 29        |
| 5.3.5    | Supplemento dei dati ottenuti tramite generic parsing . . . .                                                                | 29        |
| 5.3.6    | Fallback su Wikipedia OpenSearch API . . . . .                                                                               | 30        |
| 5.4      | Selezione dei contenuti rilevanti, generazione e gestione del contesto<br>per l'LLM . . . . .                                | 30        |
| 5.4.1    | Chunking manuale e token count . . . . .                                                                                     | 30        |
| 5.4.2    | Chunking manuale e bi-encoding, ranking mediante cosine<br>similarity . . . . .                                              | 30        |
| 5.4.3    | RecursiveCharacterTextSplitter, bi-encoding e cross-encoding                                                                 | 31        |
| 5.5      | Iniezione del <code>query_context</code> nel contesto dell'LLM e generazione<br>della risposta finale per l'utente . . . . . | 32        |
| 5.5.1    | Stima dell'affidabilità ed assegnazione da parte dell'LLM del<br>relativo score . . . . .                                    | 33        |
| 5.5.2    | Trasparenza ed enumerazione delle fonti di riferimento . . . .                                                               | 34        |
| <b>6</b> | <b>Ottimizzazioni, sicurezza e benchmark</b>                                                                                 | <b>35</b> |
| <b>7</b> | <b>Conclusione</b>                                                                                                           | <b>37</b> |
|          | <b>Bibliografia</b>                                                                                                          | <b>39</b> |

## Capitolo 5

# Implementazione della pipeline RAG

In questo capitolo tratteremo estensivamente la fase implementativa del progetto e, come preannunciato, quali siano state le principali sfide da risolvere durante lo sviluppo di quest'ultimo. Per ragioni di brevità, ci concentreremo principalmente sull'implementazione della componente essenziale, e a mio avviso maggiormente affascinante, di Sapienza-DC: la pipeline RAG. In ciascuna delle sezioni presentate, descriveremo le varie componenti della pipeline di retrieval-augmented generation, ciascuna avente un obiettivo ben preciso, evidenziando gli approcci nonché tecnologie impiegate al fine di raggiungere quest'ultimo.

### 5.1 Processamento preliminare della query dell'utente e disambiguazione

Si è pensato fin da subito di processare la richiesta dell'utente tramite un LLM (detto *guardrail*<sup>1</sup>), richiedendo a quest'ultimo, mediante un prompt adeguato, di classificare la query come **ACCEPTED** o **REJECTED**. Nel caso in cui dovesse essere rifiutata, l'utente viene notificato con il messaggio "Your query appears to be meaningless or invalid. Please ask a clear, answerable question".

In seguito è stata introdotta una terza possibile categorizzazione della query dell'utente, ossia **AMBIGUOUS**, nel caso in cui il soggetto della richiesta dovesse risultare omesso oppure dovessero essere individuate entità con molteplici significati (*i.e.* la parola "mercurio" può intendere l'elemento chimico, ma anche il pianeta). In tal caso il sistema sospende il retrieval e richiede delucidazioni all'utente tramite una query opportuna generata dinamicamente.

Nelle ultime versioni del progetto si è affidato all'LLM *guardrail* l'ulteriore compito di identificare e scartare in maniera sistematica richieste malevole da parte dell'utente, come, ad esempio, tentativi di prompt injection, *persona replacement*, e/o manipolazione emotiva.

In realtà ogni LLM della pipeline è istruito in tal senso, in modo tale da non far ricadere *in primis* la responsabilità interamente sul *guardrail* e in secondo luogo di

---

<sup>1</sup> *guardrail*

permettere esclusivamente alle query legittime, o più in generale al sottoinsieme delle richieste accettabili di queste ultime, di essere processate dal sistema e di ricevere una risposta.

## 5.2 Assegnazione della query a un dominio e fallback

Una sfida significativa consiste nel *routing* di una richiesta ricevuta. In particolare, per ogni richiesta in input alla pipeline, è necessario porsi la seguente domanda: “qual è il dominio più adatto a soddisfare la query dell’utente?”.

Non è semplice trovare un procedimento deterministico che risponda in maniera precisa a questa domanda. Pertanto, anche in questo caso ci si è affidati a un LLM (difatti il guardrail stesso) per indirizzare la query dell’utente verso il dominio ritenuto più opportuno tra i seguenti: `it.wikipedia.org`; `www.raiplaysound.it`; `www.ipsos.com`; `www.marvel.com`.

Si noti che in alcuni casi potrebbe accadere che la richiesta effettuata dall’utente non possa trovare risposta in alcuno dei domini sopra riportati. Per questo motivo, in un secondo momento, si è permesso all’LLM di decidere di assegnare un dominio generico (individuato tramite “\*”) alla query ricevuta come meccanismo di fallback.

Quando ciò accade si tenta di cercare le informazioni necessarie per rispondere alla domanda su un dominio qualunque, effettuando un parsing generico di pagine web ritenute pertinenti.

A un secondo LLM viene dunque affidato il compito di sintetizzare la domanda dell’utente in poche *keyword*<sup>2</sup> che costituiranno la query per il motore di ricerca.

```
chatbot_backend-1 | ORIGINAL LLM GUARDRAIL ANSWER: ```json
chatbot_backend-1 | {
chatbot_backend-1 |   "status": "ALLOWED",
chatbot_backend-1 |   "domain": "www.marvel.com",
chatbot_backend-1 |   "requested_information": null
chatbot_backend-1 | }
chatbot_backend-1 | ```
chatbot_backend-1 | [QueryHandler] LLM answered the prompt with the following:
chatbot_backend-1 | {'accepted': True, 'proposed_domain': 'www.marvel.com'}
chatbot_backend-1 | [QueryHandler] LLM answered the prompt with the following:
chatbot_backend-1 | {'search_query': 'chi è Spiderman', 'user_query': 'Chi è Spiderman?'}
chatbot_backend-1 | [QueryHandler] | [INFO] Reformulated user query as: 'Chi è Spiderman?'
```

**Figura 5.1.** Esempio di processamento e assegnazione della query a un dominio.

La peculiarità di questo procedimento consiste nel fatto che l’output dell’LLM dipenda anche dalla *chat history*, vale a dire la cronologia delle interazioni precedenti con l’utente.

In altre parole, se è stato richiesto all’utente un chiarimento sulla query iniziale, sia essa ad esempio “parlami di Mercurio” e l’utente ha disambiguato con “mi riferisco al pianeta”, l’LLM non proporrà come `search_query` “pianeta”, ma “pianeta mercurio”. Il sistema è pertanto in grado di mantenere il contesto nonché filo logico della conversazione, anche qualora l’utente dovesse disconnettersi, grazie alla memorizzazione persistente sul database.

A partire dalle ultime implementazioni del progetto, si è deciso di fare in modo che l’LLM, data la query in input, non ritornasse esclusivamente la singola query per

<sup>2</sup> keyword

il search engine, ma una coppia `<search_query, user_query>`, in modo tale da poter passare la `user_query` all'LLM responsabile di fornire la risposta all'utente, senza perdere informazioni sulla domanda fatta originariamente. Un esempio di ciò è facilmente consultabile nell'immagine riportata nella pagina precedente.

### 5.3 Ricerca degli URL e parsing delle pagine web

Data una query dell'utente e il dominio assegnato a quest'ultima è necessario trovare gli URL delle pagine che contengono le informazioni necessarie a rispondere ad essa.

Ci si è ben presto resi conto che ottenere in maniera programmatica i risultati di una ricerca su web da un search engine non fosse semplice. In effetti, tutte le API rese disponibili per questo scopo si sono rivelate essere a pagamento oppure con limiti di utilizzo particolarmente stringenti.

#### 5.3.1 DDGS - Duck Duck Go Search

Una prima soluzione alla quale si è giunti consisteva nell'utilizzo della libreria **DDGS**, basata sul motore di ricerca Duck Duck Go. Questa soluzione si è rivelata tutto fuorché ottimale poiché dopo poche ricerche il motore ha segnalato le nostre richieste come sospette e ha cominciato a non restituire più alcun risultato.

#### 5.3.2 SearXNG e qwant

Un tentativo successivo ha visto l'impiego di SearXNG, un *metasearch engine* open-source che aggrega risultati di ricerche web dai principali servizi di ricerca disponibili (Google, Bing, Brave, etc.). L'unico servizio di ricerca che si è rivelato essere realmente utilizzabile è stato **qwant**, dal momento che tutti gli altri motori hanno cominciato a bloccare immediatamente le richieste provenienti dal nostro IP.

Anche in questo caso, tuttavia, dopo alcuni giorni di testing, l'unico motore di ricerca ancora a nostra disposizione ha sospeso il nostro IP, lasciandoci difatto senza alcuna alternativa per la ricerca di pagine web da cui estrarre i dati per la nostra pipeline.

#### 5.3.3 Custom web scraper per Startpage.com

In seguito a un'ampia riflessione si è giunti a una nuova soluzione: parsare le pagine stesse contenenti i risultati di una determinata query effettuata su un search engine. Trovare un modo concreto di farlo si è rivelato essere non banale.

Infatti, l'analisi del DOM di una pagina restituita da una ricerca su Google ha ben presto evidenziato una problematica sostanziale: ogni singolo *div*<sup>3</sup> o *span*<sup>4</sup> contenente i risultati della ricerca era caratterizzato da classi e id generati in modo del tutto casuale per ogni nuova ricerca, rendendo impossibile estrarre i dati in modo automatizzato.

Ciononostante, dopo diverso tempo dedicato alla ricerca di possibili soluzioni, si è stati in grado di trovare un motore di ricerca privato che fa utilizzo di una ben

---

<sup>3</sup> div:

<sup>4</sup> span:

nota API di ricerca a pagamento: SerpAPI<sup>5</sup>. Tale motore di ricerca è Startpage e, al contrario di Google, non implementa meccanismi *anti-crawling*<sup>6</sup> nelle pagine restituite in seguito a una query.

Pertanto, usufruendo ancora una volta della libreria Crawl4AI e modificando leggermente il progetto dell'esonero (Minerva Web Parser) in modo tale da introdurre un nuovo parser (**StartpageSearchEngineParser**) capace di restituire, data una query, un dominio di interesse e un intero  $k$ , i primi  $k$  risultati di tale ricerca (senza duplicati) su Startpage. Inoltre, tramite l'aggiunta di un semplice endpoint `/get_query_results` abbiamo difatti creato una nostra API personalizzata al fine di permettere lo scraping e il retrieval degli URL risultanti da una ricerca su web.

Si potrebbe argomentare che i risultati restituiti possano non essere affidabili quanto quelli forniti da Google, ma la convenienza della soluzione appena presentata consiste proprio nel fatto che SerpAPI stesso ottenga i propri risultati a partire da quest'ultimo.

Si propone di seguito una porzione di codice della classe **StartpageUrlRetriever**, la quale implementa, tra le altre cose, il metodo `retrieve_relevant_urls`, che si occupa di recuperare, attraverso un'opportuna chiamata all'endpoint `/get_query_results`, esposto da MWP, i primi  $k$  URL di cui effettuare parsing al fine di ottenere informazioni rilevanti per rispondere alla query.

```

1 def retrieve_relevant_urls(
2     self,
3     search_query: str, # ricerca da effettuare
4     target_domain: str, # dominio target
5     max_results: int | None = None # numero massimo di risultati
6 ) -> list[dict[str, str]]:
7     """
8     <docstring funzione>
9     """
10    limit = max_results if max_results is not None\
11        else self.__STARTPAGE_SEARCH_RESULTS_LIMIT
12
13    generic_search: bool = (target_domain == '*')
14    # costruisce la query da effettuare con l'operatore site:
15    filtered_query = f"{search_query} site:{target_domain}"\
16        if (not generic_search)\
17        else f"{search_query}"
18
19    # estrae il risultato della ricerca da Startpage
20    data = self.mwp_client.parse_query(
21        filtered_query,
22        target_domain if (not generic_search) else "*",
23        limit)
24    scraped_urls: list[str] = []
25    if (data):
26        scraped_urls: list[str] = data
27    ...

```

Listato 5.1. Codice Python per il retrieval dei primi  $k$  URL rilevanti.

<sup>5</sup> sus

<sup>6</sup> anti-crawling:

```

chatbot_backend-1 | [QueryHandler] | [INFO] Initializing search for query: 'chi è Spiderman'
chatbot_backend-1 | [QueryHandler] | [INFO] Searching across domains: ['www.marvel.com', '*']
chatbot_backend-1 | [QueryHandler] | [INFO] Successfully scraped the following URLs to parse:
                  | ['https://www.marvel.com/characters/scarlet-spider-ben-reilly', 'https://it.wikipedia.org/wiki/Uomo_Ragno', 'https://www.esquire.com/it/cultura/film/a41866919/peter-parker/', 'https://boutique-spiderman.fr/it/pages/peter-parker-tout-savoir-sur-le-vrai-visage-de-spider-man']

```

Figura 5.2. Esempio di ricerca degli URL pertinenti alla query.

```

backend-1 | [INIT]... → Crawl4AI 0.8.6
backend-1 | [StartpageSearchEngineParser] | [INFO] Searching for 'chi è Spiderman site:www.marvel.com'.
backend-1 | [FETCH]... ↓ https://www.startpage.com
          | | ✓ | 🕒 1.21s
backend-1 | [SCRAPE]... ♦ https://www.startpage.com
          | | ✓ | 🕒 0.01s
backend-1 | [COMPLETE] • https://www.startpage.com
          | | ✓ | 🕒 1.24s
backend-1 | [StartpageSearchEngineParser] | [INFO] Parsing completed successfully.

```

Figura 5.3. Scraping degli URL tramite StartpageSearchEngineParser. (MWP)

### 5.3.4 Estrazione dei dati dagli URL selezionati

A questo punto, dati gli URL ottenuti da una query al motore di ricerca e il dominio target è immediato estrarne i contenuti in formato markdown.

Tuttavia, bisogna tenere presente che a una determinata query  $q$ , in caso di fallback, può essere assegnato un dominio generico `"*"`. Anche in questo caso abbiamo dovuto modificare leggermente il progetto di esonero, aggiungendo un secondo parser, `GenericParser`, adatto ad estrarre in modo approssimativo i contenuti di una qualunque pagina web.

```

backend-1 | [API-SERVER] | [INFO] Received parsing request for URL: https://it.wikipedia.org/wiki/Uomo_Ragno
backend-1 | [API-SERVER] | [INFO] Extracted domain from URL: it.wikipedia.org
backend-1 | [INIT]... → Crawl4AI 0.8.6
backend-1 | [FETCH]... ↓ https://it.wikipedia.org/wiki/Uomo_Ragno
          | | ✓ | 🕒 1.25s
backend-1 | [SCRAPE]... ♦ https://it.wikipedia.org/wiki/Uomo_Ragno
          | | ✓ | 🕒 0.45s
backend-1 | [COMPLETE] • https://it.wikipedia.org/wiki/Uomo_Ragno
          | | ✓ | 🕒 1.72s
backend-1 | [WikipediaParser] Original HTML file length (in characters): 1122572
backend-1 | [WikipediaParser] Successfully parsed webpage titled 'Uomo Ragno - Wikipedia' for a total of 241480 characters.

```

Figura 5.4. Estrazione dei contenuti da una pagina web, dato un URL. (MWP)

Naturalmente, come già precedentemente sottolineato, risulta impossibile effettuare quest'operazione senza introdurre una quantità minima di rumore (dati inutili) dovuto alla sostanziale differenza che si può avere tra pagine di domini diversi. Si tratta di un *trade-off*<sup>7</sup> che abbiamo ritenuto accettabile.

### 5.3.5 Supplemento dei dati ottenuti tramite generic parsing

Per permettere all'LLM che risponde alla query dell'utente di disporre della massima quantità di informazioni relative all'argomento della conversazione, ogni ricerca per il dominio  $X$  viene supplementata con i primi  $k$  risultati della stessa ricerca effettuata su domini generici, purché il dominio scelto dal guardrail non sia già `"*"`.

Questo equivale ad effettuare due query al search engine, dove la prima è `"site:X {query}"` mentre la seconda consiste semplicemente nel ricercare `"{query}"` senza utilizzare l'operatore `"site:"`.

<sup>7</sup> tradeoff

### 5.3.6 Fallback su Wikipedia OpenSearch API

## 5.4 Selezione dei contenuti rilevanti, generazione e gestione del contesto per l'LLM

In questa sezione verrà trattato il processo tramite cui si è in grado di scegliere quali testi (`parsed_text`) e quali porzioni di questi ultimi sono da considerare rilevanti e dunque da fornire all'LLM che risponde alla query dell'utente.

Si tratta fondamentalmente della sfida più ardua nella costruzione della pipeline RAG: il contesto dell'LLM non è infinito e va dunque gestito opportunamente in modo tale da non saturarlo.

Difatti, se decidessimo di fornire pagine intere come fonti di riferimento al modello, non solo rischieremmo di esaurire la context window ma introdurremmo un notevole overhead in fase di risposta, diminuendo notevolmente il throughput della pipeline.

L'idea alla base è quella di stilare una classifica dei pezzi (o chunk) di testo che contengono, con maggiore probabilità, la risposta alla domanda dell'utente.

Una volta ottenuta la classifica (o ranking) di questi ultimi, è sufficiente prendere i primi  $k$  chunk, i quali andranno a costituire il `query_context` dell'LLM, ossia l'insieme di testi di riferimento nonché informazioni che il modello utilizzerà per rispondere all'utente, purché in essi vi sia la risposta alla query.

In caso contrario, il modello risponderà semplicemente con "Mi dispiace, ma non ho abbastanza informazioni per rispondere a questa domanda".

Di seguito è proposto un resoconto degli approcci utilizzati, in fase di sviluppo del progetto, per raggiungere l'obiettivo appena descritto.

### 5.4.1 Chunking manuale e token count

Un primo approccio *naive*<sup>8</sup> è stato quello di dividere manualmente il `parsed_text` ottenuto da ciascun URL in chunk di lunghezza arbitraria.

In questa prima soluzione proposta, i chunk venivano classificati per ordine di count decrescente di `query_token` contenuti in essi.

Purtroppo, questa soluzione semplice ed estremamente conveniente comportava in realtà risultati piuttosto scarsi.

### 5.4.2 Chunking manuale e bi-encoding, ranking mediante cosine similarity

Un altro difetto del primo approccio consisteva nel fatto di tagliare arbitrariamente il chunk una volta raggiunti  $k$  caratteri. Questo poteva comportare il troncamento di paragrafi, frasi o addirittura parole.

In questa seconda iterazione di selezione abbiamo voluto rifinire il processo di chunking, tenendo conto di queste problematiche, ad esempio, aggiungendo il testo contenuto tra una newline e la successiva, fino al raggiungimento della lunghezza massima per un chunk.

---

<sup>8</sup> naive:

In realtà anche questo modo di procedere non era perfetto: due frasi separate da newline e appartenenti allo stesso paragrafo potevano essere divise in chunk diversi, comportando la dispersione di informazioni.

A questo punto sia query che chunk estratti venivano convertiti in vettori  $n$ -dimensionali, dove  $n$  dipende dal modello di embedding utilizzato.

Il modello utilizzato in questa fase è stato inizialmente `nomic-embed-text`. Ogni porzione di testo da tradurre veniva tradotta in un vettore mediante l'opportuna chiamata API.

Infine, per ciascuna coppia `<query_vec, chunk_vec>` veniva calcolata la metrica di cosine similarity, utilizzata successivamente come score per i singoli chunk. La lista di chunk veniva riordinata sulla base di questi ultimi e tutti i chunk con punteggi inferiori a una certa soglia, ossia `COSINE_SIMILARITY_THRESHOLD`, venivano automaticamente scartati.

Ora, si selezionavano i primi  $k$  nella lista e si costruiva il contesto per l'LLM, fino al raggiungimento di una lunghezza massima predefinita, cioè `MAX_OUTPUT_LENGTH`.

### 5.4.3 RecursiveCharacterTextSplitter, bi-encoding e cross-encoding

Nell'implementazione finale di questa componente della pipeline si è deciso di sostituire il chunking manuale dei testi con una funzione utility, `RecursiveCharacterTextSplitter` della libreria `LangChain`.

Tale modifica permette di suddividere in modo preciso il testo in chunk, senza perdere informazioni e mantenendo una soglia limite di  $k$  caratteri.

Ancora una volta si procede al bi-encoding di query e chunk, tuttavia con un modello più leggero (`intfloat/multilingual-e5-small`, fornito dalla libreria `TextEmbedder`) e in esecuzione direttamente nel container del backend (nell'iterazione precedente il modello si trovava in esecuzione su un container separato, introducendo maggiore overhead).

In aggiunta a ciò, `TextEmbedder` permette l'embedding di testo in *batch*, operazione indubbiamente più efficiente dell'embedding iterativo di ogni singolo chunk ottenuto dai testi estratti nella fase precedente.

Il passaggio successivo rimane invariato, difatti, una volta effettuato l'encoding si stila una lista sulla base dello score in termini di cosine similarity e si scartano tutti i chunk al di sotto della soglia stabilita.

Tuttavia, un alto score di cosine similarity garantisce solo la vicinanza semantica del chunk per cui è stato calcolato alla query dell'utente, ma non assicura che quest'ultimo contenga veramente la risposta alla domanda posta.

Per questo motivo, si è deciso di implementare un re-ranking dei primi  $k$  chunk così ottenuti per costruire la "graduatoria" finale. Si utilizza pertanto un cross-encoder (nel nostro caso `ms-marco-MiniLM-L-6-v2`), in grado di stabilire quali siano realmente i chunk contenenti le informazioni utili a rispondere alla `user_query`.

Dopo l'applicazione, anche in questo caso, di una soglia di punteggio minimo, ossia `CROSS_ENCODING_SIMILARITY_THRESHOLD`, si aggiungono i chunk al `query_context`, fino al raggiungimento della lunghezza massima per quest'ultimo. Per completezza, viene di seguito presentato la porzione di codice (contenuto nella classe `ChunkSelector`) ove viene effettuato quanto appena descritto.

```

1 def select_relevant_chunks(
2     cls,
3     query: str,
4     parsed_pages: list[tuple[str, str]]
5 ) -> dict[str, list[str]]:
6     ...
7     # primo stadio: embedding di query e chunk estratti (e5-small)
8     query_vector: list[float] = TextEmbedder.embed_batch(
9         [query],
10        query=True
11    )[0]
12    ...
13    chunk_texts = [candidate[1] for candidate in all_candidates]
14    chunk_vectors: list[list[float]] = TextEmbedder.embed_batch(
15        chunk_texts,
16        query=False
17    )
18
19    chunk_scores: list[tuple[str, str, float]] = []
20
21    for (url, chunk_text), c_vec in zip(
22        all_candidates,
23        chunk_vectors):
24
25        # calcola cosine similarity per la coppia
26        score = cls.__calculate_cosine_similarity(
27            query_vector,
28            c_vec
29        )
30
31        # filtra chunk con score insufficiente
32        if score >= cls.__COSINE_SIMILARITY_THRESHOLD:
33            chunk_scores.append((url, chunk_text, score))
34
35        # ordinamento e selezione di top-k per reranking
36        chunk_scores.sort(key=lambda x: x[2], reverse=True)
37        top_k_for_rerank = chunk_scores[:cls.__TOP_K_CHUNKS]
38
39        # secondo stadio: cross-encoder per reranking (ms-marco)
40        reranked_scores = cls.__rerank_chunks(query, top_k_for_rerank)
41        ... # costruzione contesto LLM

```

**Listato 5.2.** Codice Python che implementa la fase di two-stage chunk retrieval.

Tale implementazione assicura che, qualora le informazioni estratte, o quanto-meno una porzione di esse, dovesse contenere la risposta alla domanda dell'utente, quest'ultima sarà certamente inserita nel contesto dell'LLM, permettendo dunque a quest'ultimo di fornire la risposta corretta all'utente.

## 5.5 Iniezione del `query_context` nel contesto dell'LLM e generazione della risposta finale per l'utente

Si tratta difatti del passaggio più semplice tra tutti quelli visti finora. L'unica difficoltà consiste nell'effettuare *grounding* dell'LLM, vale a dire evitare, attraverso

*prompt engineering*, che il modello utilizzi informazioni di cui è già in possesso per rispondere alla domanda dell'utente.

L'obiettivo è proprio quello di avere un modello che sia in grado di rispondere alla `user_query` se e solo se la risposta è effettivamente contenuta nelle fonti ad esso fornite.

Ora è sufficiente iniettare la `user_query` nel prompt dell'LLM assieme al contesto della query (`query_context`) e attendere la sua risposta.

### 5.5.1 Stima dell'affidabilità ed assegnazione da parte dell'LLM del relativo score

Al termine della propria risposta, una regola ferrea stabilita in fase di prompting impone all'LLM di emettere obbligatoriamente una breve sezione denominata "affidabilità", costituita da un breve commento sulla propria risposta, in cui il modello specifica quante delle informazioni fornite derivino direttamente dal testo e quanto queste ultime siano da ritenere pertanto attendibili.

Si noti che, come già asserito, l'LLM è istruito in modo tale da utilizzare esclusivamente le informazioni direttamente ricavabili dalle fonti estratte, e non nozioni già in suo possesso. Nel seguente listato viene fornita la porzione di prompt responsabile del grounding dell'LLM, contenuto nella classe `PromptBuilder` del modulo `llm_manager`.

```

1 @staticmethod
2     def build_answer_user_query_prompt(
3         query: str,
4         query_context_data: str
5     ) -> str:
6
7     prompt = f"""
8     ...
9     1. RELIABILITY DETERMINATION (HIGHEST PRIORITY): At the end of
10    EVERY answer, UNLESS you are using the exact fallback string from
11    Rule 5, you MUST add a new line containing the reliability score.
12    - If the Question is in Italian: 'Affidabilità: <comment> (<score
13    >/5)'
14    - If the Question is in English: 'Reliability: <comment> (<score
15    >/5)'
16    The <comment> MUST briefly justify the score based on how well the
17    reference texts support the answer. [...]
18
19    2. STRICT GROUNDING: You must ONLY use facts explicitly mentioned
20    in the <reference_texts>. [...] NEVER use outside knowledge.
21
22    5. INSUFFICIENT INFO: If the <reference_texts> do not contain any
23    information to answer the question, output EXACTLY this string
24    and NOTHING ELSE:
25        - For Italian: 'Mi dispiace, ma non ho abbastanza
26        informazioni per rispondere a questa domanda.'
```

**Listato 5.3.** Porzione di prompt che gestisce il grounding dell'LLM.

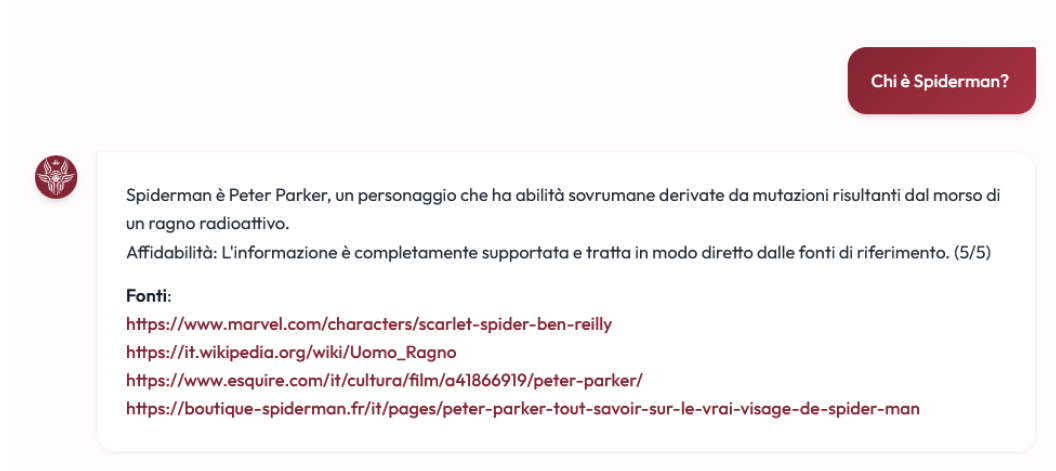
Il modello fornisce, accanto al commento, un punteggio indicativo (da 1 a 5) dell'indice di affidabilità (o *reliability*) della risposta data all'utente.

Questo garantisce *in primis* trasparenza e permette inoltre all'utente di stabilire in modo indicativo in che porzione attenersi alla risposta appena ricevuta, sebbene sia sempre opportuno che quest'ultimo verifichi indipendentemente tali informazioni.

### 5.5.2 Trasparenza ed enumerazione delle fonti di riferimento

A tal riguardo, in fondo alla risposta del modello sono sempre indicate le fonti utilizzate da quest'ultimo per rispondere alla domanda dell'utente.

Questo permette, tra le altre cose, una verifica celere di quanto riportato nella risposta generata dall'LLM.



**Figura 5.5.** La risposta finale alla query “Chi è Spiderman?”, generata dall'LLM.

Questo conclude la trattazione sulla pipeline implementata durante lo sviluppo del progetto. Per sintetizzare quanto detto nonché chiarire eventuali dubbi sulle diverse fasi previste da quest'ultima, si propone, nella pagina a seguire, un diagramma riepilogativo.

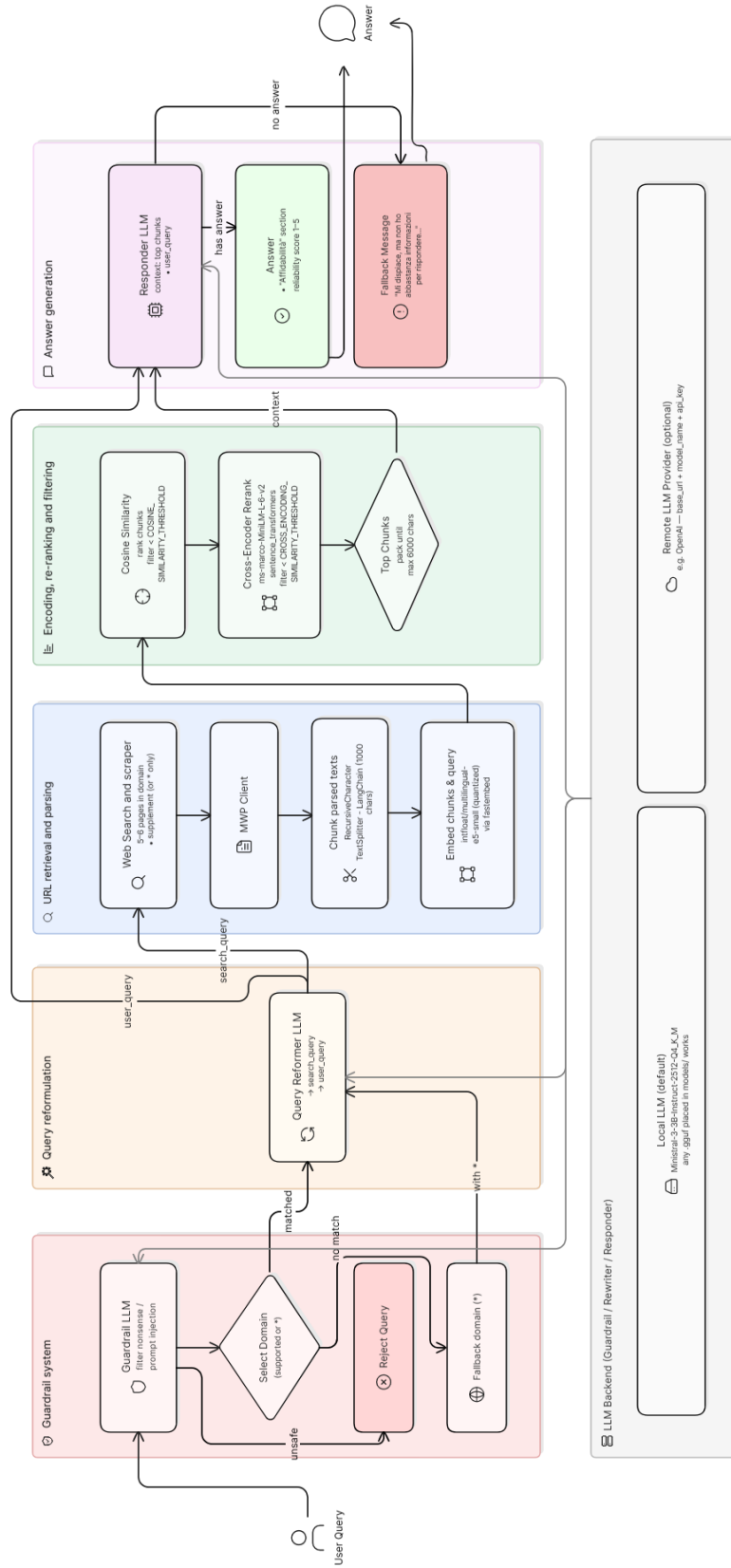


Figura 5.6. Diagramma della pipeline di retrieval-augmented generation (RAG).



# Bibliografia

- [1] Inc. Docker. Docker documentation. <https://docs.docker.com/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [2] MariaDB Foundation. Mariadb documentation. <https://mariadb.com/docs>, 2026. Ultimo accesso: 01 Giugno 2026.
- [3] Markus Heiser. Searxng documentation. <https://docs.searxng.org/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [4] Hossein. Crawl4ai documentation. <https://docs.crawl4ai.com/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [5] Michael Chiang Jeffrey Morgan. Ollama documentation. <https://ollama.readthedocs.io/en/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [6] Fabio Pastore, Yi Hao Wu, and Alessandro Zannone. Repository ufficiale di minerva web parser. <https://github.com/fabio-pastore/minerva-web-parser>, 2026. Ultimo accesso: 04 Giugno 2026.
- [7] Fabio Pastore, Yi Hao Wu, and Alessandro Zannone. Repository ufficiale di sapienza-dc. <https://github.com/fabio-pastore/sapienza-DC>, 2026. Ultimo accesso: 01 Giugno 2026.
- [8] Qdrant. Fastembed documentation. <https://qdrant.tech/documentation/fastembed/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [9] Sebastián Ramírez. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [10] Nils Reimers. Sentencetransformers documentation. <https://sbert.net/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [11] Armin Ronacher. Jinja2 documentation. <https://jinja.palletsprojects.com/en/stable/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [12] Hugging Face Team. Hugging face documentation. <https://huggingface.co/docs>, 2026. Ultimo accesso: 01 Giugno 2026.
- [13] LangChain Team. Langchain documentation. <https://docs.langchain.com/>, 2026. Ultimo accesso: 01 Giugno 2026.

- [14] MySQL Documentation Team. Mysql documentation. <https://dev.mysql.com/doc/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [15] Adam Wathan. Tailwindcss documentation. <https://v2.tailwindcss.com/docs>, 2026. Ultimo accesso: 01 Giugno 2026.